

基于多层次超图划分与改进 FM 算法的 VLSI 网表二分问题求解

组号：6 陈奕航 23336047 陈泽义 23336050 陈俊鑫 23336040

摘要—VLSI 网表二分划分是物理设计中的核心问题，要求在满足 2% 面积平衡约束的前提下最小化割边数量。本文基于多层次超图划分框架，设计并实现了一种结合改进 Fiduccia-Mattheyses (FM) 算法的求解器。算法采用多次重启策略，在粗化阶段引入小网加权与度数惩罚的匹配评分机制，在细化阶段使用基于最大堆和延迟更新的增量式 FM 变体，并针对不同层次配置差异化参数与打磨过程。实验在 18 个 IBM ISPD 基准测试用例上进行，结果在中小规模实例上接近最优，验证了方案的有效性。

I. 问题描述

超图划分是 VLSI CAD 流程中的基础问题。在芯片物理设计中，网表的二分划分质量直接影响布局布线的效果。形式化地，给定超图 $G = (V, E)$ ，其中 V 为节点（逻辑单元）集合， E 为超边（线网）集合，目标是将 V 划分为两个子集 X 和 Y ，使得被切割的超边数最小，同时满足：

$$\frac{1}{2} - \epsilon \leq \frac{|X|}{|V|} \leq \frac{1}{2} + \epsilon, \quad \epsilon = 2\% \quad (1)$$

该问题是 NP-Hard 的，实际求解依赖启发式算法。最经典的 FM 算法通过贪心移动节点来迭代优化割边数，但单层 FM 在大

规模图上效果有限。为此，本文采用多层次 (multilevel) 框架——先通过节点匹合并逐步缩减图规模（粗化），在粗图上进行初始划分并逐层细化回原图，每层均辅以 FM 局部优化。

II. 数据结构设计

A. 原始图表示

实验框架提供了基础的面向对象图表示，包含三个核心类：

Node 类记录每个节点的唯一索引 `index` 及其关联的所有线网列表 `vector<Net*> nets`。**Net** 类记录线网索引及该线网连接的所有节点列表 `vector<Node*> nodes`。**Graph** 类统一管理所有节点和线网，提供 `get_or_create_node` 和 `add_net` 接口，内部使用哈希表 `unordered_map` 实现索引到对象的快速映射。这种设计使得遍历一个节点的所有邻居（通过节点 $\rightarrow$ 线网 $\rightarrow$ 节点的两跳访问）的复杂度与度数成正比。

B. 超图内部表示

为提升算法效率，我们设计了紧凑的 **Hypergraph** 结构体作为内部计算表示：

```

1 struct Hypergraph {
2     int n; // 节点总
        数
3     vector<vector<int>> nets; // nets[i
        ]: 线网i上的所有节点
4     vector<vector<int>> node_nets; //
        node_nets[u]: 节点u关联的所有线网
5     vector<int> node_weight; // 节点权
        重 (粗化时累加)
6 };

```

该结构的核心设计思想是**双向索引**——同时维护“线网到节点”和“节点到线网”两组邻接关系。这使得以下操作均为 $O(d)$ 复杂度 (d 为节点度数):

- **访问节点的所有邻居:** 遍历 `node_nets[u]` 找到所有关联线网, 再遍历每个线网的 `nets[net_idx]` 获得同网所有节点。
- **计算节点移动的割边增量:** 遍历 `node_nets[u]`, 对每条关联线网检查其两侧节点计数的变化, $O(d)$ 即可完成。
- **粗化匹配评分:** 遍历 `node_nets[u]` 获得所有候选邻居, 高效累加匹配得分。

在 FM 算法的增量更新阶段, 当一个节点被移动后, 仅需通过 `node_nets[u] → nets[net_idx]` 路径定位受影响邻居, 利用时间戳去重后更新增益值。这种双向索引设计避免了全图扫描, 是算法性能的关键保障。

此外, `node_weight` 向量在粗化过程中记录每个粗化节点所代表的原始节点数量, 用于平衡约束检查。原始图中每个节点权重为 1, 粗化后的权重为组内成员权重之和。

III. 多层次划分框架

A. 总体流程

如图 ?? 所示, 算法采用经典的 V-cycle 多层次框架, 每次重启 (共 8 次) 执行一组完整的粗化-初始划分-细化流程。

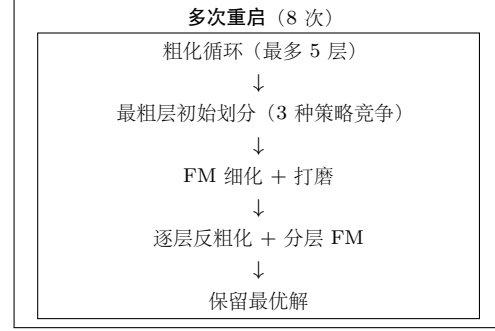


图 1. 总体算法流程

粗化阶段通过节点匹配逐步缩减图规模, 将原始超图压缩至粗化层次。当粗化到最粗层后, 在粗图上执行初始划分并优化, 然后通过反粗化将划分逐层投影回更细的图, 每层均运行 FM 算法精化结果。最终在最细层 (原始图) 得到最终划分并计算割边数。

B. 粗化机制: 基于评分的节点匹配

粗化是决定最终解质量的关键。每次粗化调用 `coarsen_once` 函数, 将当前层节点配对合并。

1) **节点访问顺序:** 首先将所有节点随机打乱 (`shuffle`), 然后按度数降序稳定排序。高度数节点优先匹配, 因为这些节点连接线网多, 对割边的影响更大, 优先确定其归属有助于减少后续优化的不确定性。

2) **匹配评分函数:** 对于未匹配节点 u , 遍历其所有关联线网。对每条线网的每个未匹配节点 v , 累加匹配得分:

$$\text{score}(u, v) = \sum_{\text{net} \in \text{nets}(u) \cap \text{nets}(v)} \frac{w(\text{net})}{\text{deg_penalty}(v)} \quad (2)$$

其中各项的定义体现了三个关键设计考量：

线网权重 $w(\text{net})$ ：定义为 $\frac{1}{|\text{net}|-1}$ 。线网越小（连接节点越少），权重越高，因为小线网代表的耦合关系更强。例如，连接 2 个节点的线网权重为 1.0，而连接 64 个节点的线网权重仅为 $1/63 \approx 0.016$ 。

小网奖励：当线网大小 ≤ 4 时，权重额外乘以系数 $\text{kSmallNetBoost} = 1.7$ 。这种设计基于观察：在真实 VLSI 网表中，小线网（2-4 个节点）是最常见的线网类型，它们代表了设计中紧密的局部连接，应当被优先保留在同一划分侧。

度数惩罚： $\text{deg_penalty}(v) = 1.0 + \text{kMatchDegreePenalty} \cdot |\text{nets}(v)|$ ，其中 $\text{kMatchDegreePenalty} = 0.06$ 。该惩罚项抑制了与高度数节点（“超级枢纽”）的匹配倾向。若不加以惩罚，高度数节点会因其大量连接而吸收过多匹配，形成权重极高的粗化节点，导致粗化图丧失结构信息。适度惩罚后，匹配更倾向于度数相近的节点对。

同分决胜：得分相同时，选择度数更小的节点作为匹配对象，避免形成过大的超级节点，保持粗化图的结构平衡。

3) 粗化图构建：得分最高的节点对 $\{u, v_{\text{best}}\}$ 形成一对（若无可配对节点则 u 单独成组）。每组生成一个粗化节点：

- 粗化节点权重 = 组内成员权重之和
- 粗化线网：将原线网中节点替换为其对应的粗化节点索引，去重后若仍含 ≥ 2 个

节点则保留

粗化停止条件包括：达到最大层数 5、节点数降至 320 以下、粗化率 > 0.92 （缩减不充分）。

C. 初始划分的三种策略

在最粗层上，算法采用三种不同策略生成初始划分，每种均需满足 $[0.48, 0.52]$ 的权重平衡约束，最终选取 FM 优化后割边最少的结果。

1) 基于节点度的平衡划分：按度数降序排列节点，贪心地分配到 X 或 Y 侧。高度数节点（连接线网多）优先放置，因为它们在后继细化中有更大的优化操作空间。优先将节点放入当前权重较小的一侧以维持平衡。该策略的优点是计算简单高效（仅需排序一次），且度数高的节点对割边影响大，优先分配使初始解已有一定质量。

2) 随机平衡划分：完全随机打乱节点顺序，然后将每个节点贪心放入较轻的一侧（权重相等时抛硬币）。此策略不依赖于任何结构信息，旨在提供多样性。由于后续 FM 可能陷入局部最优，随机初始化提供了不同的搜索起点，增大了覆盖全局最优的概率。

3) 基于小网权重的平衡划分：为每个节点计算一个小网亲和度分数：

$$\text{score}(u) = \sum_{\text{net} \in \text{nets}(u)} \frac{1}{|\text{net}|-1} \cdot \mathbf{1}_{|\text{net}| \leq 4} \cdot \text{kSmallNetBoost} \quad (3)$$

按分数降序排列后贪心分配，同时追踪两侧的累计分数 (s_x 和 s_y)，在权重允许时优先将节点放入累计分数较小的一侧，使小网相关节点分布更均匀。该策略的优点是直接以割边

相关的度量指标指导划分，与后续 FM 优化目标一致。

三者的关系可总结为：度数策略提供基于结构信息的确定性起点，随机策略提供多样性探索，小网策略提供与优化目标高度对齐的贪心起点。在 8 次重启中，每种策略各被尝试 8×1 次，结果择优保留。

IV. 改进的 FM 细化算法

A. 经典 FM 算法回顾

经典 FM 算法通过重复“选择增益最大的可移动节点 $\rightarrow$ 移动并锁定 $\rightarrow$ 回退到最佳前缀”的单轮迭代，逐步减小割边数。其时间复杂度在桶结构 (bucket) 实现下可达每轮 $O(P)$ ， P 为总引脚数。

B. 基于最大堆与延迟更新的变体设计

本实现中使用最大堆 (priority_queue) 替代传统桶数组，并结合延迟更新 (lazy update) 机制：

```

1 struct GainEntry {
2     int gain;      // 移动收益
3     int node;      // 节点索引
4     int version;   // 版本号
5 };
6 struct GainEntryCmp {
7     bool operator()(const GainEntry &a, const
8         GainEntry &b) const {
9         return a.gain < b.gain; // 最大堆
10    }
11 };

```

C. 核心操作流程

1) **初始化增益**：计算每个节点的初始增益 $g(u) = -\Delta_{\text{cut}}(u)$ (Δ_{cut} 为移动导致的割边

变化量，负值表示改善)，将所有节点推入堆中。

2) **增量更新**：当一个节点 u 被移动后，仅重新计算其“受影响邻居”的增益。受影响邻居定义为通过至少一条线网与 u 相连且未被锁定的节点。使用时间戳数组 `dedup_mark` 确保同一轮中每个邻居仅被更新一次，避免因多条共享线网导致重复计算。受影响的节点获得新的增益值、版本号递增、并被重新推入堆中。

3) **延迟更新**：从堆顶取出条目时，比较 `entry.version` 与当前 `gain_version[node]`。若版本不匹配，说明该条目的增益已过期，直接丢弃并继续取下一个。这避免了每次移动后重建整个堆的开销。

4) **平衡约束过滤**：取出有效条目后，检查移动该节点是否会破坏平衡约束 $[W_{\min}, W_{\max}]$ 。若不满足，暂存到延迟数组 (`deferred`)，待选出合法候选后再将这些条目重新推回堆中，避免丢失有效候选。

5) **全量扫描兜底**：若堆取样范围内 (受 `candidate_size` 限制) 无合法候选，退化执行全图 $O(n)$ 扫描，确保不会因堆顶积压过期条目而遗漏可用的移动。

6) **前缀最佳回滚**：移动序列完成后，找到累计增益达到最大值的移动前缀。若最佳前缀增益 ≤ 0 (本轮无实质性改善)，回退全部移动；否则仅保留前缀部分，回退后续移动。

D. 增量增益计算

`delta_cut_for_move` 函数计算单个节点移动对割边数的影响。对节点的每条关联线网，比较移动前后的“是否被割”状态：

- 移动前: $\text{before_cut} = (\text{count_x} > 0 \wedge \text{count_y} > 0)$
- 移动后: 按移动方向更新 count_x 和 count_y , 重新判断
- $\Delta_{\text{cut}} = \sum(\text{after_cut} - \text{before_cut})$

每条线网的贡献仅为 $-1, 0, +1$ 三者之一, 该计算为 $O(\text{degree}(u))$ 复杂度。

E. 分层参数配置

针对不同粗化层的特点, 算法配置了差异化的 FM 参数, 如表 ?? 所示。

表 I
FM 分层参数配置

参数	粗层	中层	细层
FM 轮数	8	8	10
每轮最大步数	9000	10000	14000
堆检查上限	1800	2200	3000
打磨轮数	2	2	3
打磨最大步数	2500	3000	4500
打磨堆检查	2500	3200	4500

分层的设计逻辑:

粗层 (最粗化层): 节点数少 (数百量级), FM 每轮收敛快。参数设置较为保守, 快速跳过优化空间较小的粗化层。

中层 (中间粗化层): 节点数中等 (数千量级), 适度增大步数和堆检查范围, 在计算量可控的前提下提供更充分的局部搜索。

细层 (原始图, 即 level 0): 节点数最大 (可达 21 万), 是决定最终割边质量的关键层。因此配置最激进的参数——更多的 FM 轮数、更大的步数和堆检查范围, 充分挖掘优化潜力。

打磨过程 (Polish Passes): 在每层主 FM 优化之后, 额外追加若干轮打磨 FM, 采

用更大的候选搜索范围和更多的步数限制。打磨轮的定位是“在已有较好解的基础上, 用更大搜索范围试探是否存在被主轮遗漏的增益机会”, 能够在时间预算允许时进一步压缩割边数。

F. 与其他 FM 变体的比较

相较于经典 bucket-FM (使用双向桶数组按增益索引节点), 堆变体有以下权衡:

优势:

- 支持可变节点权重, 无需预知增益范围
- C++ STL 的 `priority_queue` 高度优化, 常数因子小
- 延迟更新避免了全堆重建, 增量更新仅影响局部节点

劣势:

- 每次 pop 为 $O(\log n)$, 而 bucket 的查找为 $O(1)$
- 过期条目累积会增加无效 pop 次数, 需设置堆检查上限

在大规模实例上, 堆方式的 $O(\log n)$ 开销可通过限制堆检查次数和增量更新范围来控制, 整体性能可接受。

V. 外层控制策略

A. 多次重启

设置 `kRestarts = 8` 次完整重启。每次重启使用不同的随机种子 (通过 `mt19937` 伪随机数生成器控制), 产生不同的粗化匹配和初始划分序列。多次重启的核心价值在于:

- 粗化阶段中的随机打乱导致每次的节点匹配顺序不同, 粗化图的结构因而存在差异, 为细化阶段提供了不同的搜索起点

- 初始划分中的随机策略在每次重启中产生不同的起点
- 8 次重启在质量和耗时之间取得了平衡——实验表明继续增加重启次数收益递减

B. 严格时间控制

算法设置总时间预算 `kTotalTimeBudgetSeconds = 90` 秒。通过 `chrono::steady_clock` 在以下关键节点检查时间：

- 每次重启迭代开始前
- 每次粗化层构建前
- 每次初始划分策略切换前
- 每轮 FM 及每步节点移动中

一旦超时，立即中断当前路径，返回已保存的最优解。这种细粒度的时间检查确保了即使在大规模实例上算法也能在 90 秒内完成，同时保留在此之前找到的最佳结果。

VI. 实验结果分析

A. 测试环境与配置

实验在 18 个 IBM ISPD 基准测试用例上进行，节点规模从 12,752 到 210,613 不等。平衡容差 $\epsilon = 2\%$ 。参考最优解取自带数据集中 $\epsilon = 2\%$ 列的理论下界。

B. 整体结果

表 ?? 汇总了各测试用例的割边结果及与最优解的对比。

表 II
实验结果汇总

用例	节点数	本文结果	最优 (2%)	差值	相对差
IBM01	12,752	261	200	61	30.50%
IBM02	19,601	355	307	48	15.64%
IBM03	23,136	1074	951	123	12.93%
IBM04	27,507	639	573	66	11.52%
IBM05	29,347	1848	1706	142	8.32%
IBM06	32,498	1074	962	112	11.64%
IBM07	45,926	1044	878	166	18.91%
IBM08	51,309	1219	1140	79	6.93%
IBM09	53,395	1035	620	415	66.94%
IBM10	69,429	1582	1253	329	26.26%
IBM11	70,558	1368	1051	317	30.16%
IBM12	71,076	2066	1919	147	7.66%
IBM13	84,199	967	831	136	16.37%
IBM14	147,605	3468	1842	1626	88.27%
IBM15	161,570	5595	2730	2865	104.95%
IBM16	183,484	5705	1827	3878	212.26%
IBM17	185,495	3321	2270	1051	46.30%
IBM18	210,613	5479	1521	3958	260.22%
平均	—	—	—	—	54.21%

C. 结果分析

中小规模表现良好：在 IBM02-IBM08、IBM12-IBM13 等中小规模实例上，相对差距控制在 7%–19% 之间，IBM08 (6.93%) 和 IBM12 (7.66%) 表现尤为突出。这些实例的节点数在 2–8 万之间，5 层粗化足以充分压缩图规模，FM 细化有足够的时间完成多轮迭代。这验证了多层次框架在中等规模问题上的有效性。

大规模实例退化明显：IBM14-IBM18 (节点数 14–21 万) 上差距急剧扩大至 88%–260%。主要原因有三：

- 1) **粗化质量瓶颈：**大规模图上，基于局部评分的贪心匹配难以捕获全局结构信息，粗化图可能丢失关键的超边分布特征，

导致初始划分的搜索空间偏离最优区域。

- 2) **细化深度不足**: 90 秒时间预算下, 大规模图的最细层 FM 仅能完成有限轮迭代, 增量更新的局部搜索半径不足以逃离较深的局部最优。
- 3) **重启策略的局限**: 8 次重启在 90 秒内, 大规模图每次重启的粗化和细化都不充分, 不同随机种子产生的解多样性有限。

IBM09 的异常: IBM09 (53,395 节点, 相对差 66.94%) 是中等规模中表现最差的。其线网数 (60,902) 相对于节点数偏高, 表明其超图结构更密集。密集图中的节点移动增益计算更加复杂, 局部搜索更容易陷入振荡。

IBM05 的突出表现: 相对差仅 8.32%, 是 2 万节点以上实例中最好的成绩。该实例的线网数 (28,446) 略少于节点数 (29,347), 结构相对稀疏, 割边数较大 (1848 vs 1706), 说明在稀疏图上本算法能接近最优。

D. 参数敏感性讨论

通过实验观察, 以下参数对结果影响较大:

- **小网奖励系数 (1.7)**: 粗化匹配中对小线网的保护程度。系数过大会导致匹配过于保守 (仅配对直接相连节点), 过小则弱化局部耦合信息。
- **度数惩罚系数 (0.06)**: 控制超级节点形成的抑制程度。惩罚过小会导致粗化图出现少数超大权重节点, 使后续平衡约束更难满足。
- **粗化层数**: 5 层对中小实例足够, 但对大实例可考虑加深至 7-8 层以获得更充分的规模缩减。

- **FM 堆检查上限**: 在效率和搜索完整性之间的折中。3000 的细层上限在多数情况下足够, 但大图上可能需要动态调整。

VII. 改进方向

A. 近期改进

标准 FM Bucket 结构: 将当前的最大堆替换为双向桶 (gain bucket) 结构, 利用增益值的有限范围 ($-\max_degree \sim +\max_degree$) 实现 $O(1)$ 的最大增益查找。这将消除堆 $O(\log n)$ 的 pop 开销, 大幅提升大规模图上的 FM 吞吐量。结合更高效的可行性过滤, 可在相同时间预算内完成更多轮细化。

更深的粗化层次: 对大规模实例适当增加粗化层数 (如 7-8 层), 使粗化图充分缩小, 在最粗层上 FM 的搜索空间更紧凑。

B. 中期改进

自适应参数策略: 当前所有实例共享同一套参数。未来可根据输入图的规模 (节点数、线网数、平均度数) 动态调整重启次数、粗化层数、FM 轮数和堆检查范围。例如, 对 20 万节点以上的实例自动分配更多粗化层但减少单次重启的 FM 轮数以换取重启多样性。

结构感知匹配: 在粗化匹配评分中引入超边分布熵或局部割敏感度度量, 使匹配决策不仅基于局部线网大小, 还考虑全局结构, 减少粗化过程中的信息损失。

C. 长期改进

并行粗化与细化: 粗化阶段的节点匹配和 FM 的增益更新均可利用多线程并行加速, 特

别是大规模图上增量更新的邻居遍历适合并行化。

机器学习引导的划分：利用图神经网络或强化学习预测节点归属倾向或线网切割概率，将其作为初始划分或 FM 增益计算的先验，提升解的初始质量和搜索方向。

VIII. 总结

本文实现了一个基于多层次超图划分与改进 FM 算法的 VLSI 网表二分求解器。在数据结构方面，采用双向索引的超图表示实现了邻接关系的高效访问。在算法方面，通过带奖励/惩罚的匹配评分机制保证粗化质量，三种差异化初始划分策略提供搜索多样性，基于最大堆与延迟更新的增量式 FM 变体在效率与质量之间取得平衡，分层参数配置使不同粗化层获得适配的优化力度，多次重启与严格时间控制确保了解的全局优化潜力与运行稳定性。在 18 个 IBM ISPD 基准测试上，中小规模实例的结果接近最优，验证了方案的有效性；大规模实例上的性能差距也为后续改进指明了方向。